Université Abdelmalek Essaâdi
Faculté des Sciences et Techniques de Tanger
Département Génie Informatique
Cycle Ingénieur — Logiciels et Systèmes Intelligents (LSI)

Module JEE — Compte rendu d'atelier

Application web de gestion de commandes basée sur MVC et les standards JEE

Servlet, JSP, JSTL, JavaBean, CDI et JDBC
sur Tomcat 9 et MySQL

Réalisé par : Sara OUHADDOU
Encadrant : Pr. Lotfi ELAACHAK
Année universitaire : 2026–2027
Dépôt du code : `https://github.com/ouhaddousara/jee-order-management-mvc`

Résumé

Ce rapport présente la réalisation de l'atelier « Application web basée sur MVC et standards JEE ». L'objectif est de maîtriser les standards *Servlet*, *JSP* et *JavaBean* en construisant une application qui gère les commandes de plusieurs clients.

Le travail suit les cinq étapes du sujet : modélisation (diagramme de classes), traduction en base MySQL, crátion du projet Maven sous Eclipse, couche de persistance (entités, DAO, interfaces de service) puis couche web (servlets et pages JSP pour les espaces client, produit et commande). Au-delà du sujet, l'application protège la cohérence du stock par des transactions, vérifie les saisies, se protège contre les attaques CSRF et XSS et propose une interface responsive.

Mots-clés : JEE, Servlet, JSP, JSTL, MVC, JDBC, DAO, CDI, MySQL, Tomcat, Maven.

Table des matières

Résumé **1**

1 Introduction **7**
1.1 Contexte et objectif 7
1.2 Outils et versions 7
1.3 Note sur le terme « MVC 1 » 8
1.4 Organisation du rapport 8

2 Étape 1 — Conception du modèle **9**
2.1 Diagramme de classes 9
2.2 Décisions de conception 10
2.3 Cycle de vie d'une commande 10

3 Étape 2 — Base de données MySQL **11**
3.1 Du diagramme au schéma relationnel 11
3.2 Contraintes portées par la base 12
3.3 Mise en place et vérification 13

4 Étape 3 — Projet Web dynamique et Maven **15**
4.1 Environnement 15
4.2 Création et conversion du projet 16

5 Étape 4 — Couche de persistance **17**
5.1 Organisation en paquets 17
5.2 Gestion des connexions 17
5.3 Transactions 17
5.4 Cohérence du stock 18
5.5 Règle de transition de statut 18
5.6 Traduction des erreurs de la base 19

6 Étape 5 — Couche web **20**
6.1 Architecture 20
6.2 Servlets 20

6.3 Choix de conception côté web 20
6.4 Pages JSP et JSTL 21
6.5 Interface 21
6.5.1 Tableau de bord 22
6.5.2 Espace client 22
6.5.3 Espace produit 23
6.5.4 Espace commande 24
6.5.5 Gestion des erreurs et mobile 27

7 Validation **29**
7.1 Stratégie 29
7.2 Preuve de la gestion du stock 30
7.3 Démarrage et déploiement 31

8 Gestion du code avec Git et GitHub **33**

9 Limites et perspectives **35**

Conclusion **36**

A Structure du dépôt **38**

Table des figures

2.1 Diagramme de classes du domaine 9
2.2 Transitions autorisées entre statuts (les états gris sont terminaux) 10

3.1 Schéma relationnel (les flèches vont de la clé étrangère vers la table référencée) . 12
3.2 Tables créées dans la base `gestion_commandes` 13
3.3 Structure des tables `commande` et `ligne_commande` 13
3.4 Données d'exemple insérées 13
3.5 Une contrainte `CHECK` refuse un stock négatif (erreur 3819) 14

4.1 Versions des outils installés 15
4.2 JDK 17 configuré dans Eclipse 15
4.3 Dépendances Maven résolues dans Eclipse 16

6.1 Circulation d'une requête dans l'application 20
6.2 Tableau de bord 22
6.3 Liste des clients 22
6.4 Formulaire client avec erreur de validation 23
6.5 Liste des produits, avec indication de rupture ou de stock bas 23
6.6 Formulaire produit 24
6.7 Liste des commandes avec filtre par statut 24
6.8 Création d'une commande avec plusieurs lignes 25
6.9 Commande en attente : lignes modifiables 25
6.10 Commande validée : lignes verrouillées, expédition possible 26
6.11 Commande annulée 26
6.12 Message affiché lorsque la quantité dépasse le stock 27
6.13 Page d'erreur pour une adresse ou un enregistrement inexistant 27
6.14 Page d'une commande sur téléphone (largeur 390 px) 28

7.1 Stock avant la commande 30
7.2 Stock après une commande de deux unités 30
7.3 Stock après annulation de la commande 31
7.4 Serveur Tomcat démarré dans Eclipse 31
7.5 Console au démarrage : initialisation de Weld puis démarrage du serveur 32

8.1 Historique des commits 33
8.2 Envoi des commits vers GitHub 33
8.3 Page du dépôt GitHub 34
8.4 Liste des commits sur GitHub 34

Liste des tableaux

1.1 Outils utilisés 7

3.1 Traduction du modèle objet vers le modèle relationnel 11

4.1 Dépendances Maven 16

5.1 Paquets de la couche persistance 17
5.2 Erreurs de la base traduites en messages utilisateur 19

6.1 Servlets et actions (la partie après le / désigne l'action) 20

7.1 Scénarios de validation 29

Chapitre 1

Introduction

1.1 Contexte et objectif

L'atelier demande de développer une application web de gestion de commandes en respectant le patron de conception MVC. Il s'agit de manipuler les briques de base de la plateforme Java EE, sans framework de haut niveau : servlets pour le contrôle, JSP pour l'affichage, JavaBeans pour le modèle, JDBC pour l'accès aux données.

L'application permet de :

— gérer des **clients** et des **produits** (création, consultation, modification, suppression) ;

— créer des **commandes** contenant plusieurs lignes, les modifier tant qu'elles sont en attente, et suivre leur cycle de vie jusqu'à l'expédition ;

— garder un **stock** exact malgré les commandes, les modifications et les annulations.

1.2 Outils et versions

Table 1.1. Outils utilisés

Outil	Rôle	Version
JDK (Eclipse Adoptium)	Compilation et exécution	17
Eclipse IDE for Enterprise Java and Web Developers	Environnement de développement	2026
Apache Maven	Gestion des dépendances et du build	3.9
Apache Tomcat	Conteneur de servlets (Servlet 4.0, JSP 2.3)	9.0.123
MySQL Server	Base de données relationnelle	8.x
Weld (CDI)	Injection de dépendances (`@Inject`, `@ApplicationScoped`)	3.1.9
Git et GitHub	Gestion de versions	–

Tomcat 9 est imposé : Tomcat 10 et suivants utilisent l'espace de noms `jakarta.*` et ne sont pas compatibles avec les dépendances `javax.*` du sujet.

1.3 Note sur le terme « MVC 1 »

Le sujet parle de « MVC 1 ». Dans la littérature, le *Model 1* désigne une architecture où les pages JSP reçoivent elles-mêmes les requêtes et appellent le modèle. Dans cet atelier, nous avons placé des **servlets** comme contrôleurs devant les JSP (architecture souvent appelée *Model 2*), car c'est ce qui correspond au rôle demandé pour les servlets et ce qui permet de séparer vraiment la logique de la présentation. Les JSP ne contiennent aucun code Java : le fichier `web.xml` interdit les scriptlets.

1.4 Organisation du rapport

Les chapitres suivent les étapes du sujet. Le chapitre 7 présente la validation, le chapitre 8 la gestion du code, et le dernier chapitre les limites de la solution.

Chapitre 2

Étape 1 — Conception du modèle

2.1 Diagramme de classes

Le domaine comporte les quatre classes demandées (`Client`, `Commande`, `Produit`, `LigneDeCommande`) auxquelles nous avons ajouté une énumération `StatutCommande`. Le statut est un ensemble fermé de valeurs ; l'exprimer en type évite les fautes de frappe et permet de centraliser les règles de transition.

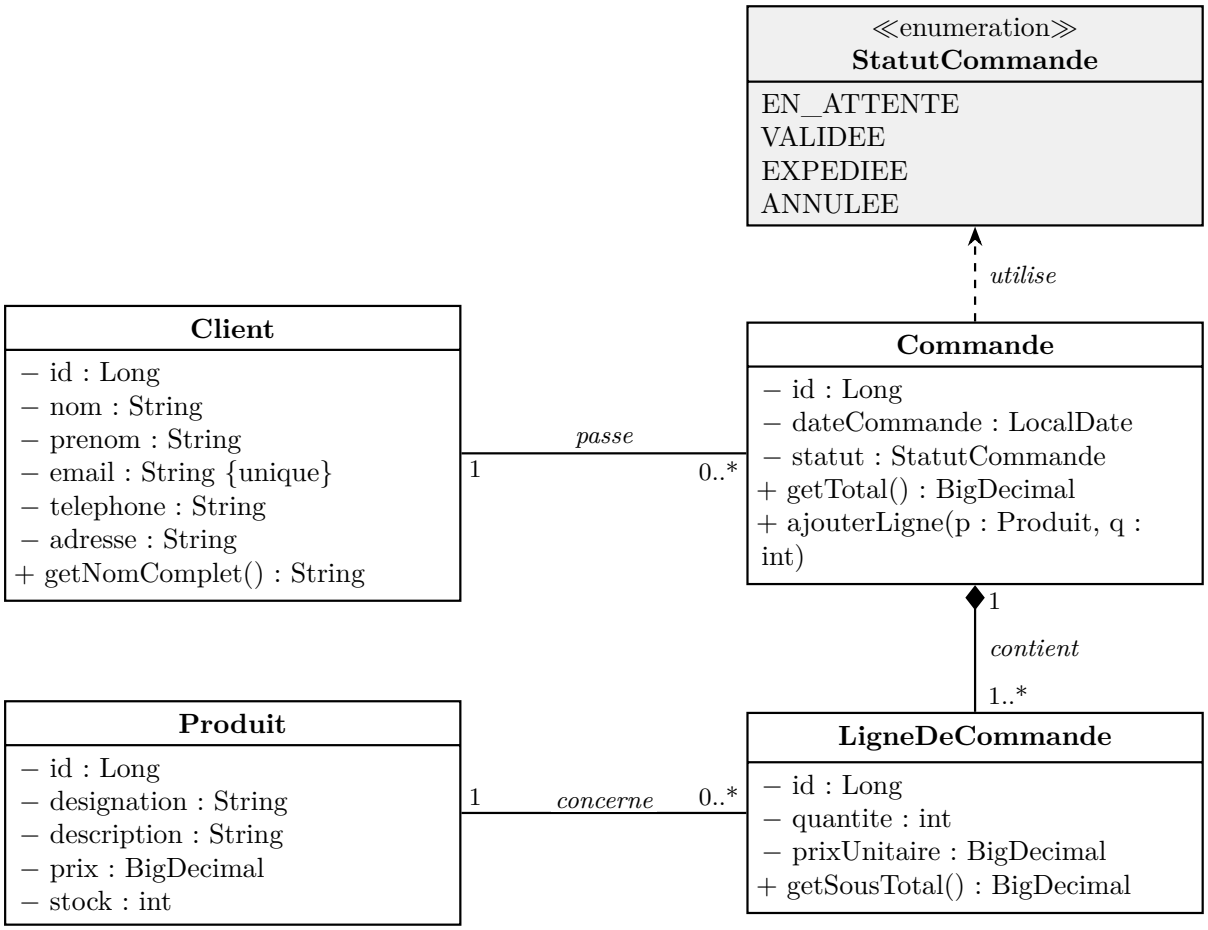


Figure 2.1. Diagramme de classes du domaine

2.2 Décisions de conception

— **`LigneDeCommande` est une classe à part entière**, et non une simple liste de produits. Une commande contient un produit *avec une quantité et un prix*. La relation entre `Commande` et `Produit` est donc une association plusieurs-à-plusieurs portée par cette classe.

— **Le prix est copié dans la ligne** (`prixUnitaire`) au moment de la commande. Si le prix d'un produit change plus tard, l'historique des commandes reste exact.

— **La composition** entre `Commande` et `LigneDeCommande` traduit que les lignes n'ont pas de sens sans leur commande.

— **Les montants sont des `BigDecimal`** : les types `double` ou `float` produisent des erreurs d'arrondi inacceptables pour de l'argent.

— **Les totaux sont calculés**, pas stockés : `getSousTotal()` et `getTotal()` dérivent des lignes, ce qui supprime tout risque d'incohérence.

2.3 Cycle de vie d'une commande

Une commande passe par quatre statuts. Les transitions autorisées sont portées par l'énumération elle-même (méthode `peutPasserA`), et non par les servlets, pour que la règle existe à un seul endroit.

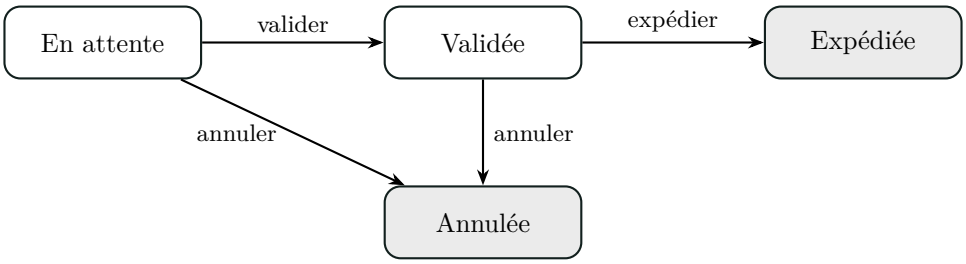


Figure 2.2. Transitions autorisées entre statuts (les états gris sont terminaux)

Les règles métier associées sont :

1. le stock d'un produit est **réservé** dès qu'une ligne est créée et **restitué** si la commande est annulée ou supprimée ;
2. les lignes ne sont modifiables que tant que la commande est *En attente*, et une commande ne peut pas perdre sa dernière ligne ;
3. une commande ne peut être supprimée que si elle est *En attente* ou *Annulée* ;
4. un client qui a des commandes ne peut pas être supprimé, ni un produit qui figure dans une commande.

Chapitre 3

Étape 2 — Base de données MySQL

3.1 Du diagramme au schéma relationnel

Chaque classe devient une table, chaque association une clé étrangère (tableau 3.1).

Table 3.1. Traduction du modèle objet vers le modèle relationnel

Élément du diagramme	**Traduction**
Classe `Client`, `Produit`	Tables `client`, `produit`
Association `Client` 1 — 0..* `Commande`	Clé étrangère `commande.client_id`, `ON DELETE RESTRICT`
Composition `Commande` 1 — 1..* `LigneDeCommande`	Table `ligne_commande`, clé étrangère `commande_id`, `ON DELETE CASCADE`
Association `LigneDeCommande` 0..* — 1 `Produit`	Clé étrangère `produit_id`, `ON DELETE RESTRICT`
Énumération `StatutCommande`	Colonne `VARCHAR` avec contrainte `CHECK` sur les quatre valeurs
`email` unique	Contrainte `UNIQUE`
Une ligne par produit et par commande	Contrainte `UNIQUE (commande_id, produit_id)`

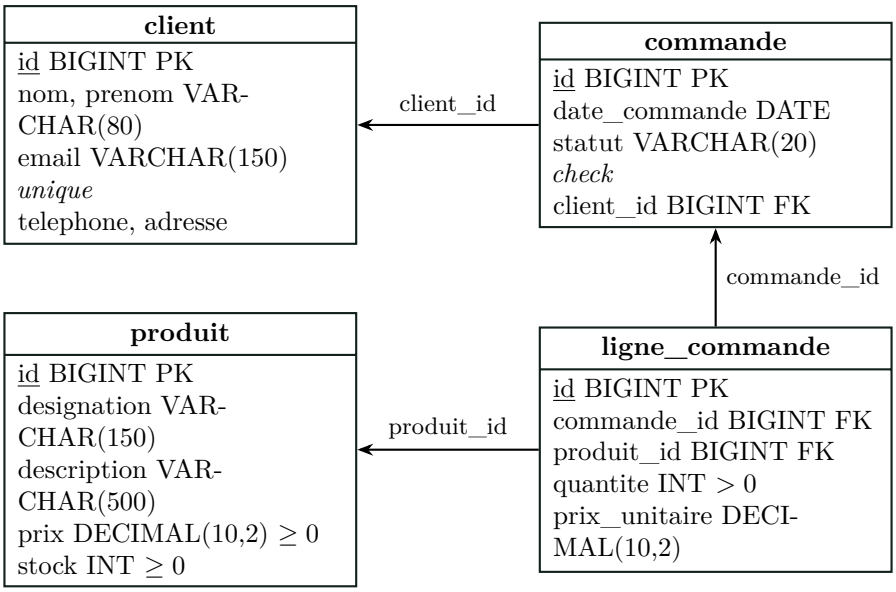


Figure 3.1. Schéma relationnel (les flèches vont de la clé étrangère vers la table référencée)

3.2 Contraintes portées par la base

Les règles essentielles sont aussi déclarées dans la base, pas seulement dans le code Java : si une autre application ou une requête manuelle écrit dans la base, les données restent cohérentes. Le code suivant montre les tables les plus riches en contraintes.

```sql
CREATE TABLE commande (
    id              BIGINT AUTO_INCREMENT PRIMARY KEY,
    date_commande   DATE        NOT NULL DEFAULT (CURRENT_DATE),
    statut          VARCHAR(20) NOT NULL DEFAULT 'EN_ATTENTE',
    client_id       BIGINT      NOT NULL,
    CONSTRAINT fk_commande_client
        FOREIGN KEY (client_id) REFERENCES client (id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT ck_commande_statut
        CHECK (statut IN ('EN_ATTENTE','VALIDEE','EXPEDIEE','ANNULEE'))
) ENGINE = InnoDB;

CREATE TABLE ligne_commande (
    id              BIGINT AUTO_INCREMENT PRIMARY KEY,
    commande_id     BIGINT        NOT NULL,
    produit_id      BIGINT        NOT NULL,
    quantite        INT           NOT NULL,
    prix_unitaire   DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_ligne_commande FOREIGN KEY (commande_id)
        REFERENCES commande (id) ON DELETE CASCADE,
    CONSTRAINT fk_ligne_produit FOREIGN KEY (produit_id)
        REFERENCES produit (id) ON DELETE RESTRICT,
    CONSTRAINT uq_ligne_commande_produit UNIQUE (commande_id, produit_id),
    CONSTRAINT ck_ligne_quantite CHECK (quantite > 0)
) ENGINE = InnoDB;
```

Listing 3.1. Extrait de `db/schema.sql` : commandes et lignes

Les choix `RESTRICT` et `CASCADE` suivent la nature des liens : un client ou un produit déjà utilisé ne doit pas disparaître silencieusement, alors que les lignes d'une commande supprimée n'ont plus de raison d'exister. Les contraintes `CHECK` ne sont appliquées qu'à partir de MySQL 8.0.16 ; avant cette version elles sont ignorées sans erreur, d'où la vérification ci-dessous.

3.3 Mise en place et vérification

Le script est exécuté avec `mysql -u root -p < db/schema.sql`. Les figures 3.2 à 3.4 montrent les tables créées, leur structure et les données d'exemple.

```
PS C:\dev\gestion-commandes> mysql -u root -p -e "USE gestion_command
es; SHOW TABLES;"
Enter password: ****************
+------------------------------+
| Tables_in_gestion_commandes |
+------------------------------+
| client                       |
| commande                     |
| ligne_commande               |
| produit                      |
+------------------------------+
```

Figure 3.2. Tables créées dans la base `gestion_commandes`

```
PS C:\dev\gestion-commandes> mysql -u root -p gestion_commandes -e "DESCRIBE commande; DESCRIBE ligne_commande;"
Enter password: ****************
+--------------+-------------+------+-----+------------+-------------------+
| Field        | Type        | Null | Key | Default    | Extra             |
+--------------+-------------+------+-----+------------+-------------------+
| id           | bigint      | NO   | PRI | NULL       | auto_increment    |
| date_commande | date       | NO   |     | curdate()  | DEFAULT_GENERATED |
| statut       | varchar(20) | NO   |     | EN_ATTENTE |                   |
| client_id    | bigint      | NO   | MUL | NULL       |                   |
+--------------+-------------+------+-----+------------+-------------------+
+---------------+---------------+------+-----+---------+----------------+
| Field         | Type          | Null | Key | Default | Extra          |
+---------------+---------------+------+-----+---------+----------------+
| id            | bigint        | NO   | PRI | NULL    | auto_increment |
| commande_id   | bigint        | NO   | MUL | NULL    |                |
| produit_id    | bigint        | NO   | MUL | NULL    |                |
| quantite      | int           | NO   |     | NULL    |                |
| prix_unitaire | decimal(10,2) | NO   |     | NULL    |                |
+---------------+---------------+------+-----+---------+----------------+
```

Figure 3.3. Structure des tables `commande` et `ligne_commande`

```
PS C:\dev\gestion-commandes> mysql -u root -p gestion_commandes -e "S
ELECT id, designation, prix, stock FROM produit; SELECT id, client_id
, date_commande, statut FROM commande;"
Enter password: ****************
+----+--------------------+---------+-------+
| id | designation        | prix    | stock |
+----+--------------------+---------+-------+
|  1 | Clavier mecanique  |  450.00 |    25 |
|  2 | Souris sans fil    |  180.00 |    40 |
|  3 | Ecran 24 pouces    | 1600.00 |    10 |
+----+--------------------+---------+-------+
+----+-----------+---------------+---------+
| id | client_id | date_commande | statut  |
+----+-----------+---------------+---------+
|  1 |         1 | 2026-10-20    | VALIDEE |
+----+-----------+---------------+---------+
```

Figure 3.4. Données d'exemple insérées

Pour s'assurer que les contraintes `CHECK` sont bien actives, nous avons tenté d'insérer un produit au stock négatif. La base refuse l'insertion (figure 3.5).

```
PS C:\dev\gestion-commandes> mysql -u root -p gestion_commandes -e "I
NSERT INTO produit (designation, prix, stock) VALUES ('Test', 10, -1)
;"
Enter password: ****************
ERROR 3819 (HY000) at line 1: Check constraint 'ck_produit_stock' is
violated.
```

Figure 3.5. Une contrainte `CHECK` refuse un stock négatif (erreur 3819)

Chapitre 4

Étape 3 — Projet Web dynamique et Maven

4.1 Environnement

La figure 4.1 récapitule les versions installées, et la figure 4.2 montre le JDK 17 configuré dans Eclipse. Une erreur rencontrée au démarrage : Eclipse signalait un niveau de conformité du compilateur trop élevé. Fixer explicitement le niveau à 17, à la fois dans Eclipse et dans le `pom.xml` (`maven.compiler.release`), garantit que le projet compile de la même manière partout.

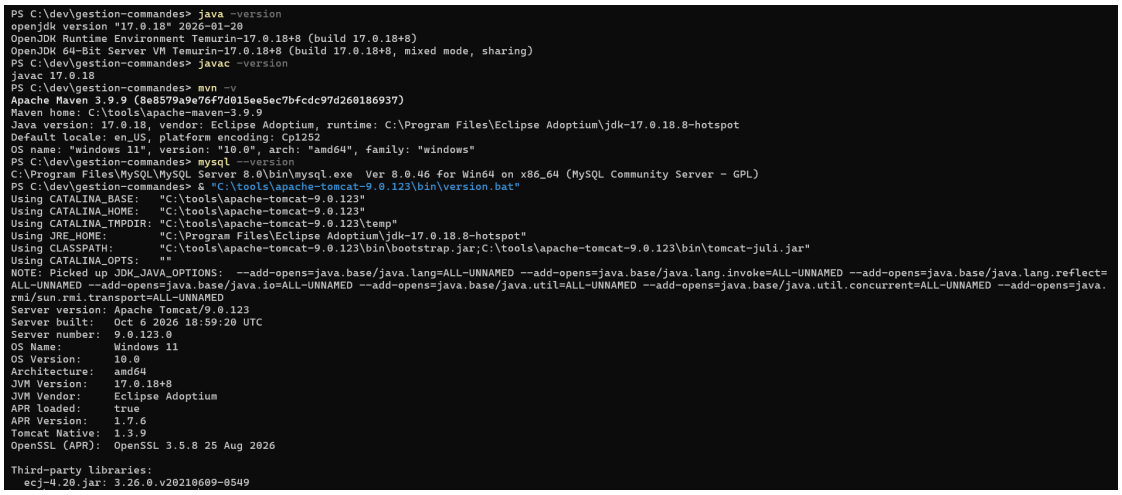


Figure 4.1. Versions des outils installés

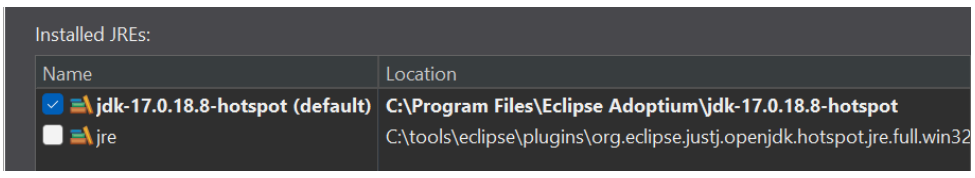


Figure 4.2. JDK 17 configuré dans Eclipse

4.2 Création et conversion du projet

Le projet a été créé comme *Dynamic Web Project* (module web 4.0), puis converti en projet Maven (*Configure* → *Convert to Maven Project*). Le `pom.xml` a ensuite été remplacé, suivi d'un *Maven* → *Update Project*. Le tableau 4.1 liste les dépendances et la raison de leur portée.

Table 4.1. Dépendances Maven

Dépendance	Portée	Raison
`javax.servlet-api` 4.0.1	provided	API Servlet, fournie par Tomcat
`javax.servlet.jsp-api` 2.3.3	provided	API JSP, fournie par Tomcat
`jstl` 1.2	compile	Bibliothèque de balises utilisée dans les JSP
`mysql-connector-j` 8.0.33	compile	Pilote JDBC pour MySQL
`weld-servlet-shaded` 3.1.9	compile	Implémentation CDI embarquée dans l'application
`javax.inject` 1	provided	Annotation `@Inject`
`jakarta.enterprise.cdi-api` 2.0.2	provided	Annotation `@ApplicationScoped`

La portée `provided` indique que Tomcat fournit déjà la bibliothèque à l'exécution : l'embarquer dans le WAR créerait des conflits de versions. Tomcat ne fournit pas CDI, d'où l'ajout de Weld. La figure 4.3 montre que les dépendances ont bien été résolues (elles apparaissent sous *Java Resources* → *Libraries*).

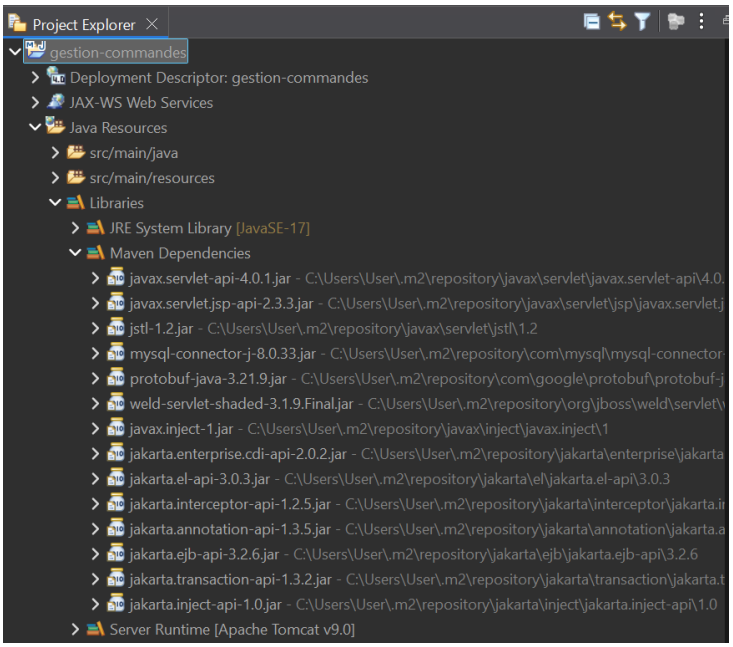


Figure 4.3. Dépendances Maven résolues dans Eclipse

Chapitre 5

Étape 4 — Couche de persistance

5.1 Organisation en paquets

Table 5.1. Paquets de la couche persistance

Paquet	Contenu
`ma.fstt.entities`	JavaBeans du modèle : `Client`, `Produit`, `Commande`, `LigneDeCommande`, `StatutCommande`
`ma.fstt.service`	Interfaces `IDao<T>`, `IClientDao`, `IProduitDao`, `ICommandeDao`, `ILigneCommandeDao` et exceptions métier
`ma.fstt.dao`	Implémentations JDBC, connexion et utilitaires transactionnels

Les servlets ne connaissent que les **interfaces** du paquet `service`. Les classes DAO sont injectées par CDI : on peut remplacer l'accès JDBC par un autre mécanisme sans modifier les contrôleurs, ni tester un contrôleur avec une fausse implémentation.

5.2 Gestion des connexions

La classe `DBConnection`, de portée `@ApplicationScoped`, lit sa configuration dans `db.properties` (ignoré par Git, un modèle est fourni) ou dans des variables d'environnement. Le mot de passe n'est donc jamais dans le dépôt.

5.3 Transactions

Une commande touche plusieurs tables (en-tête, lignes, stock). Toutes ces écritures doivent réussir ensemble ou échouer ensemble. Un utilitaire centralise cette logique pour éviter de répéter les blocs `try/catch/rollback` dans chaque méthode.

```
static <T> T write(DBConnection db, String action, SqlWork<T> work) {
    try (Connection c = db.getConnection()) {
        c.setAutoCommit(false);
        try {
            T result = work.run(c);
            c.commit();
            return result;
        } catch (SQLException | RuntimeException e) {
```

```
            rollbackQuietly(c, e);   // never leave a half-written order
            throw e;
        }
    } catch (SQLException e) {
        throw translate(action, e);
    }
}
```

Listing 5.1. Exécution d'un travail dans une transaction (`Jdbc.write`)

5.4 Cohérence du stock

Le risque classique est le suivant : deux clients commandent la dernière unité en même temps. Si le code lit le stock, vérifie puis écrit, les deux vérifications peuvent réussir. La solution retenue délègue le contrôle à la base, dans la même instruction que la modification :

```
static void adjustStock(Connection c, long produitId, int delta) throws SQLException {
    String sql = "UPDATE produit SET stock = stock + ? WHERE id = ? AND stock + ? >= 0";
    try (PreparedStatement ps = c.prepareStatement(sql)) {
        ps.setInt(1, delta);
        ps.setLong(2, produitId);
        ps.setInt(3, delta);
        if (ps.executeUpdate() == 0) {
            throw new InsufficientStockException(describeShortage(c, produitId));
        }
    }
}
```

Listing 5.2. Réservation atomique du stock

Si aucune ligne n'est modifiée, le stock était insuffisant : le message d'erreur nomme le produit et le stock réellement disponible. Deux autres protections complètent ce mécanisme :

— les lignes d'une commande sont traitées **triées par identifiant de produit**, de sorte que deux commandes simultanées verrouillent les produits dans le même ordre et ne peuvent pas se bloquer mutuellement (interblocage) ;

— l'en-tête de la commande est verrouillé (`SELECT ... FOR UPDATE`) avant toute modification de ses lignes ou de son statut.

Le **prix de la ligne** est lu dans la base au moment de l'ajout : la valeur envoyée par le navigateur est ignorée, ce qui empêche de commander à un prix falsifié.

5.5 Règle de transition de statut

```
public boolean peutPasserA(StatutCommande cible) {
    switch (this) {
        case EN_ATTENTE:
            return cible == VALIDEE || cible == ANNULEE;
        case VALIDEE:
            return cible == EXPEDIEE || cible == ANNULEE;
        default:
            return false; // EXPEDIEE and ANNULEE are terminal
    }
}
```

Listing 5.3. Transitions autorisées (`StatutCommande`)

5.6 Traduction des erreurs de la base

Les codes d'erreur MySQL sont convertis en exceptions métier avec un message lisible pour l'utilisateur (tableau 5.2). L'interface n'affiche donc jamais une trace technique.

Table 5.2. Erreurs de la base traduites en messages utilisateur

Code	Cause	Exception et sens
1062	Valeur dupliquée	`IntegrityViolationException` : e-mail déjà utilisé
1451	Ligne référencée	`IntegrityViolationException` : suppression impossible, données encore utilisées
1452	Référence absente	`IntegrityViolationException` : client, produit ou commande inexistant
3819 / 4025	Contrainte `CHECK`	`IntegrityViolationException` : prix, stock ou quantité invalide
—	Stock insuffisant	`InsufficientStockException` (levée par le code)

Chapitre 6

Étape 5 — Couche web

6.1 Architecture

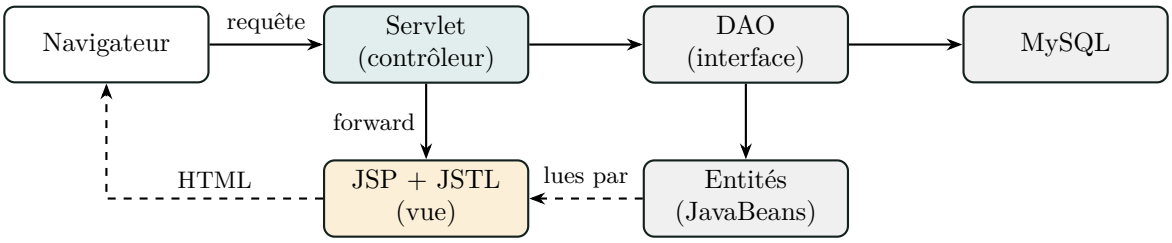


Figure 6.1. Circulation d'une requête dans l'application

Une requête arrive à un servlet, qui lit et valide les paramètres, appelle l'interface DAO, place le résultat dans la requête puis transmet (`forward`) à une JSP qui produit le HTML. Les JSP sont placées sous `WEB-INF/views` : elles ne sont pas accessibles directement depuis le navigateur, seul un servlet peut les afficher.

6.2 Servlets

Table 6.1. Servlets et actions (la partie après le `/` désigne l'action)

Servlet	Chemin	Actions
`HomeServlet`	`/home`	Tableau de bord : chiffres clés, dernières commandes, stock à surveiller
`ClientServlet`	`/clients/*`	GET : liste, `new`, `edit`, `view` ; POST : enregistrer, `delete`
`ProduitServlet`	`/produits/*`	GET : liste, `new`, `edit` ; POST : enregistrer, `delete`
`CommandeServlet`	`/commandes/*`	GET : liste (filtre par statut), `new`, `edit`, `view` ; POST : créer, modifier, supprimer, changer le statut, ajouter / modifier / retirer une ligne

6.3 Choix de conception côté web

— **Post/Redirect/Get** : tout POST réussi se termine par une redirection. Si l'utilisateur rafraîchit la page, le navigateur rejoue seulement un GET et la commande n'est pas créée

deux fois. Le message de confirmation est conservé dans la session pour une seule lecture (message *flash*).

— **Validation côté serveur** : e-mail, téléphone, prix (deux décimales au plus), stock, quantités. En cas d'erreur, le formulaire est réaffiché avec les valeurs saisies et un message par champ (figure 6.4).

— **Protection CSRF** : un filtre génère un jeton par session et refuse (erreur 403) tout POST qui ne le renvoie pas. Un site tiers ne peut pas lire ce jeton, donc ne peut pas faire supprimer une commande à la place de l'utilisateur.

— **Protection XSS** : toutes les valeurs affichées passent par `<c:out>`, qui échappe le HTML. Un client nommé `<script>` s'affiche comme du texte.

— **Encodage UTF-8** appliqué par le filtre, pour que les accents survivent de la saisie à la base et au retour.

— **En-têtes de sécurité** (`X-Frame-Options`, `X-Content-Type-Options`) et cookie de session `HttpOnly`.

```
if ("POST".equals(req.getMethod())) {
    String expected = (String) session.getAttribute("csrfToken");
    String received = req.getParameter("_csrf");
    boolean valid = expected != null && received != null
            && MessageDigest.isEqual(expected.getBytes(UTF_8), received.getBytes(UTF_8));
    if (!valid) {
        resp.sendError(HttpServletResponse.SC_FORBIDDEN);   // 403
        return;
    }
}
```

Listing 6.1. Principe de la vérification du jeton CSRF (`RequestFilter`)

La comparaison utilise `MessageDigest.isEqual`, dont la durée ne dépend pas de l'endroit où les deux chaînes diffèrent, ce qui évite de révéler le jeton par mesure de temps.

6.4 Pages JSP et JSTL

Conformément à la consigne, les pages utilisent exclusivement JSTL et le langage d'expression : `c:forEach`, `c:if`, `c:choose`, `c:out` et `fmt`. Le fichier `web.xml` déclare `scripting-invalid`, ce qui fait échouer toute page contenant un scriptlet. Le code répétitif est factorisé en *tag files* dans `WEB-INF/tags` :

— `layout.tag` : gabarit commun (menu, message flash, pied de page) ;

— `money.tag` : montant avec deux décimales et la devise « MAD » ;

— `statut.tag` : statut d'une commande avec sa couleur ;

— `csrf.tag` : champ caché du jeton, placé dans chaque formulaire.

6.5 Interface

L'interface n'utilise aucune bibliothèque externe : un seul fichier CSS et un petit script JavaScript facultatif (confirmations, ajout de lignes). Toutes les pages fonctionnent sans JavaScript. Le menu est latéral sur ordinateur et passe en barre supérieure sur téléphone. Le statut d'une commande

est indiqué par un point de couleur suivi du libellé, et la page d'une commande affiche une frise d'avancement en trois étapes.

6.5.1 Tableau de bord

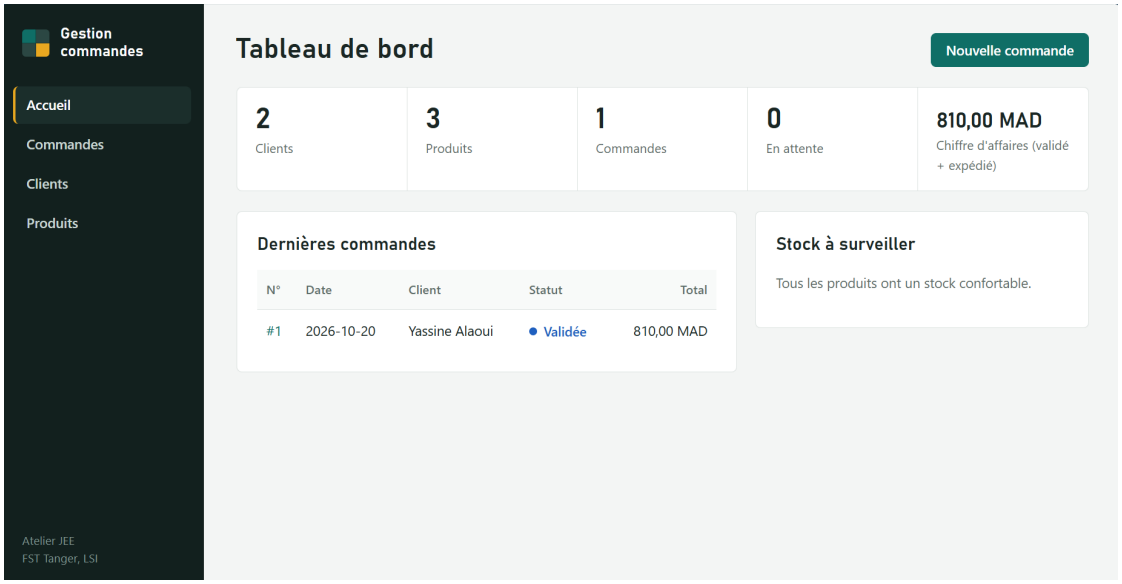


Figure 6.2. Tableau de bord

6.5.2 Espace client

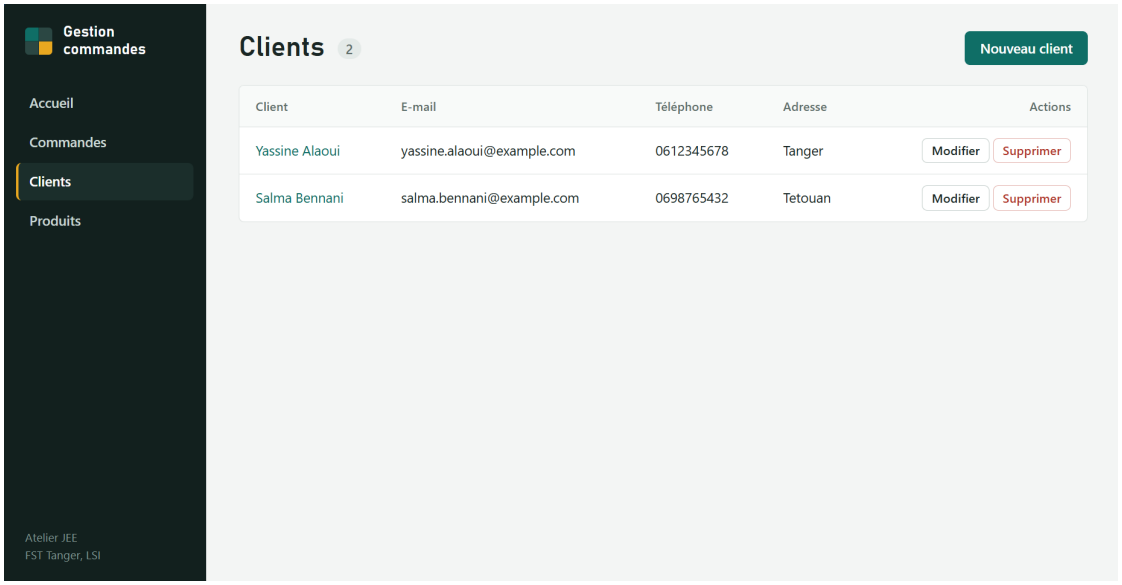


Figure 6.3. Liste des clients

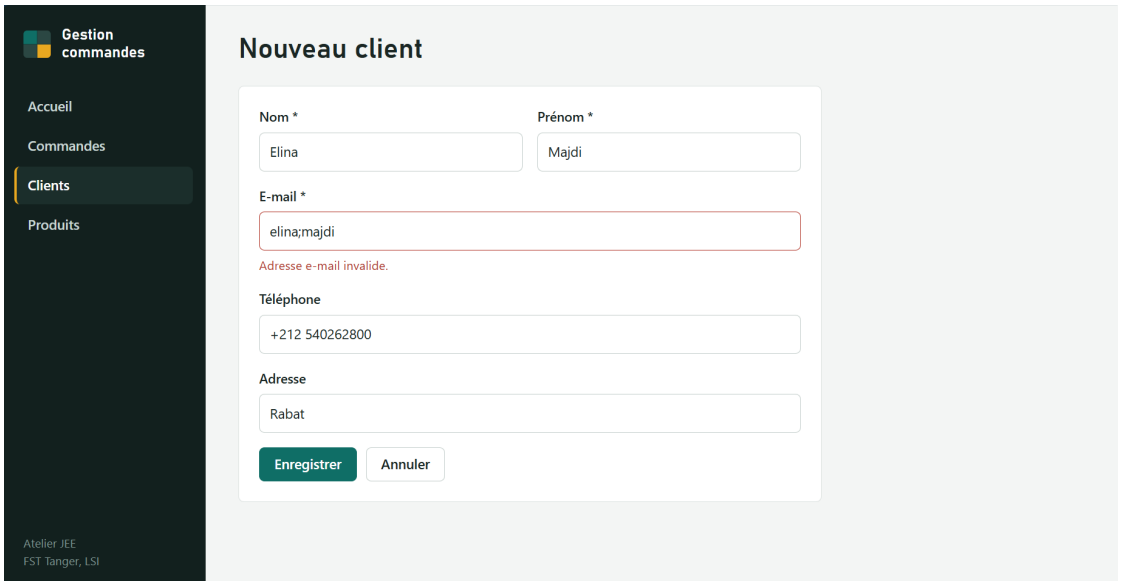


Figure 6.4. Formulaire client avec erreur de validation

6.5.3 Espace produit

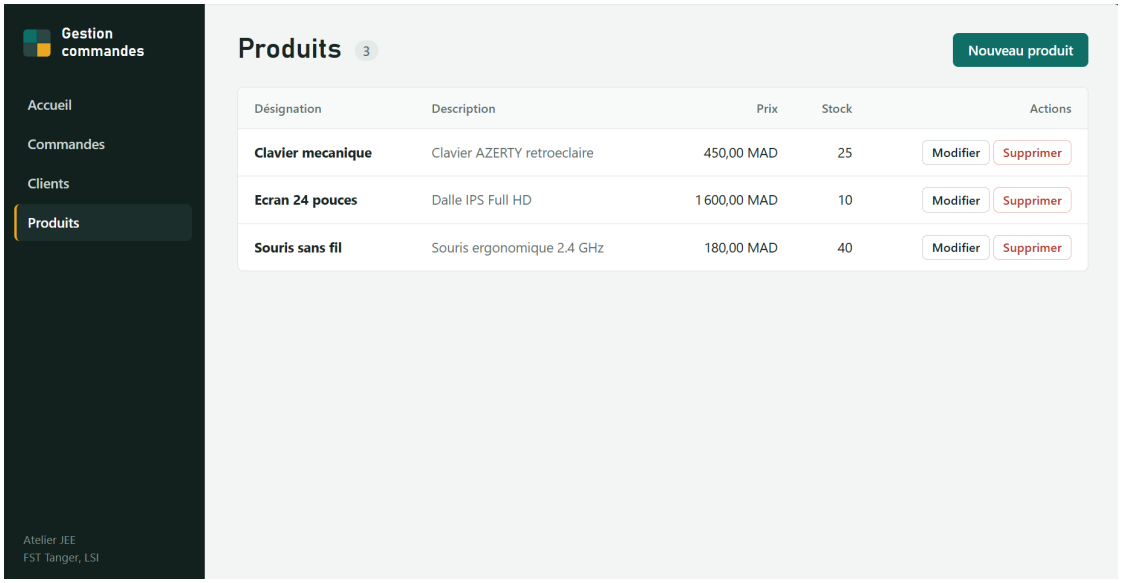


Figure 6.5. Liste des produits, avec indication de rupture ou de stock bas

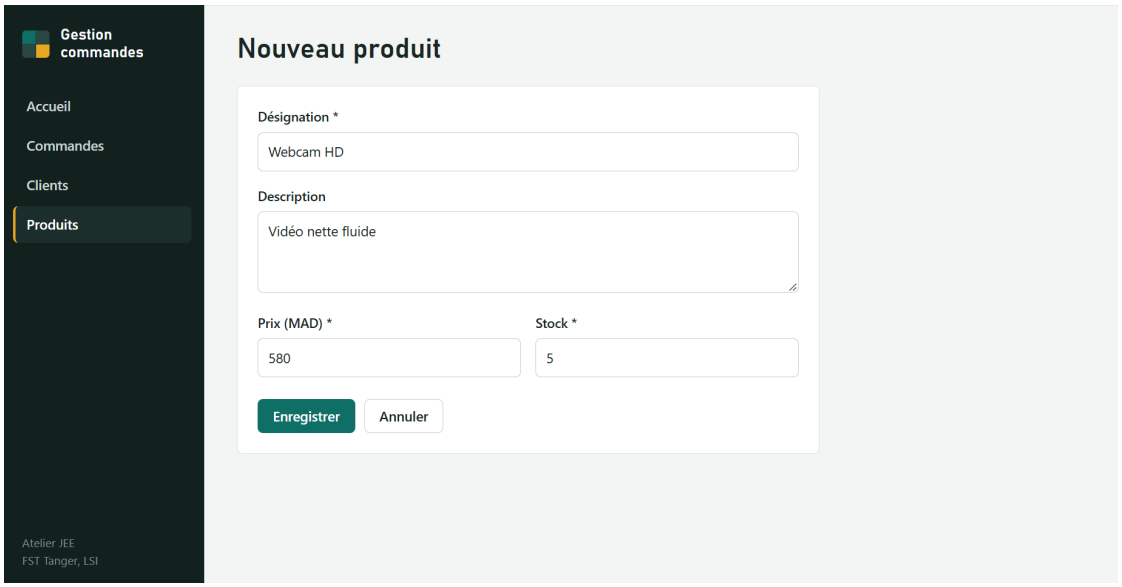


Figure 6.6. Formulaire produit

6.5.4 Espace commande

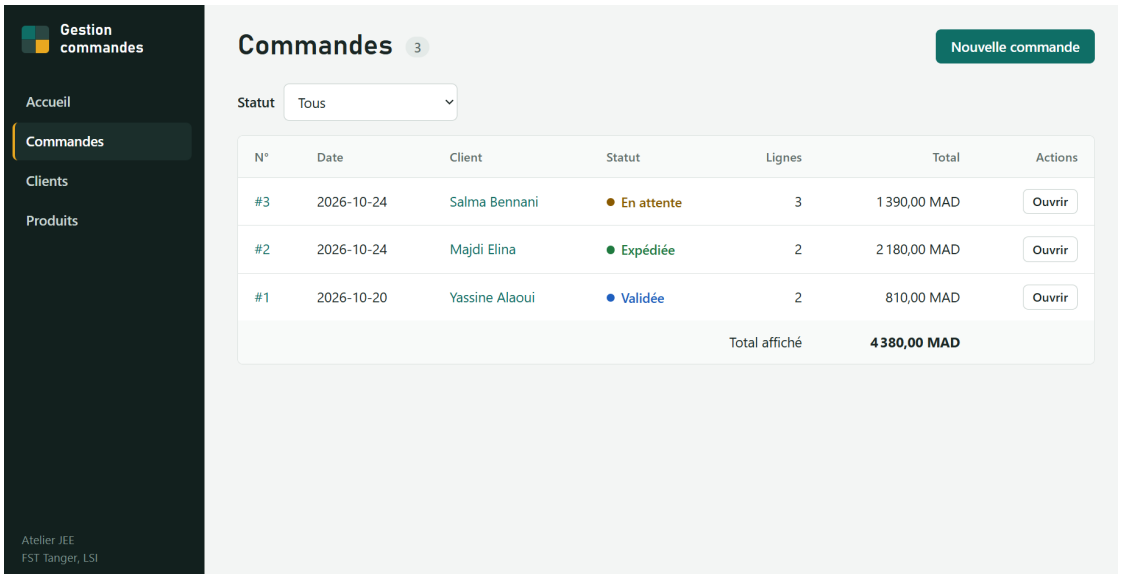


Figure 6.7. Liste des commandes avec filtre par statut

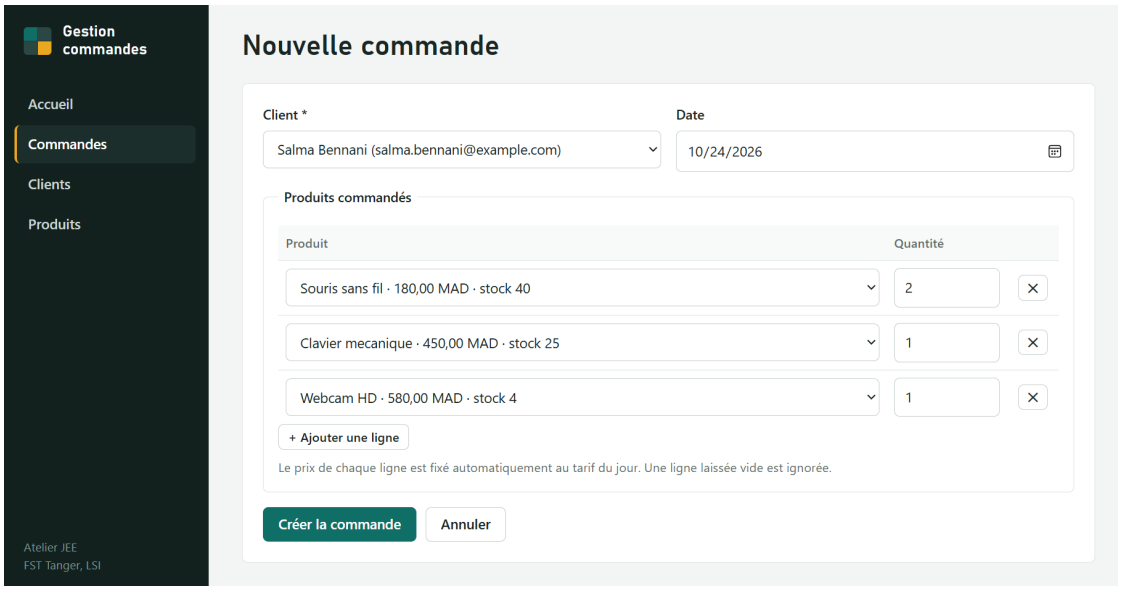


Figure 6.8. Création d'une commande avec plusieurs lignes

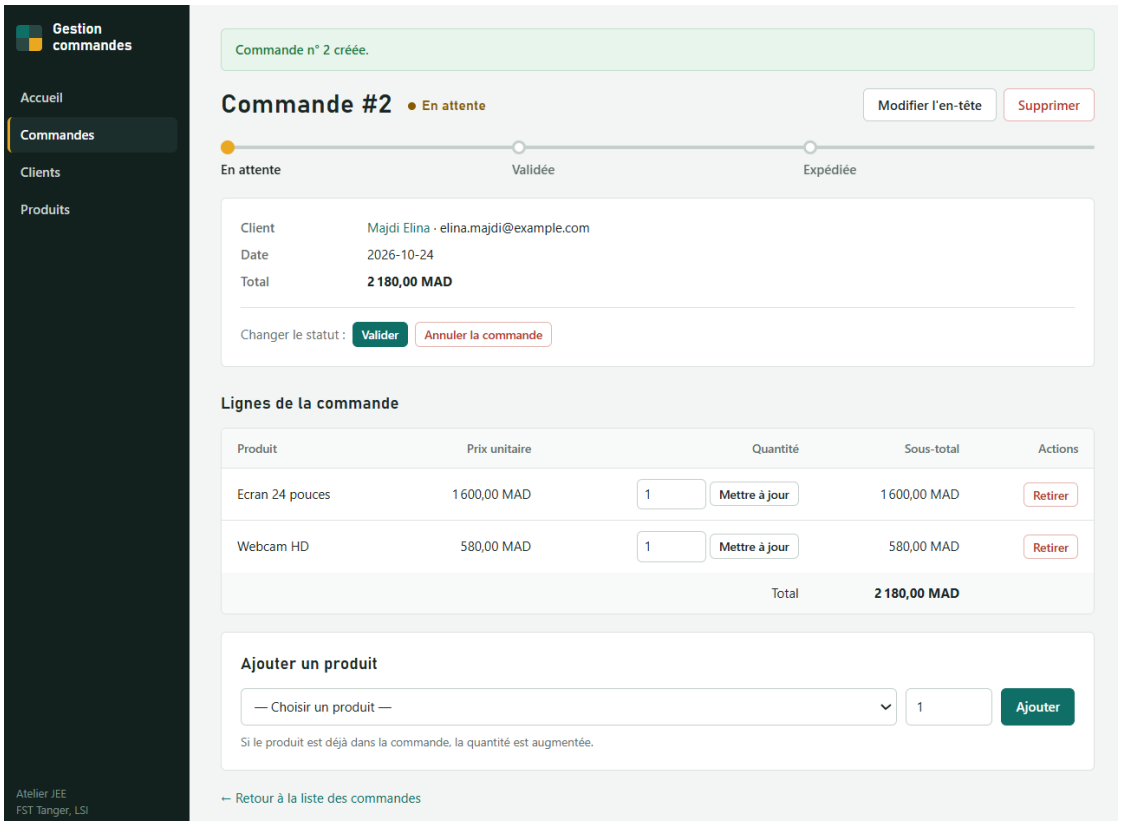


Figure 6.9. Commande en attente : lignes modifiables

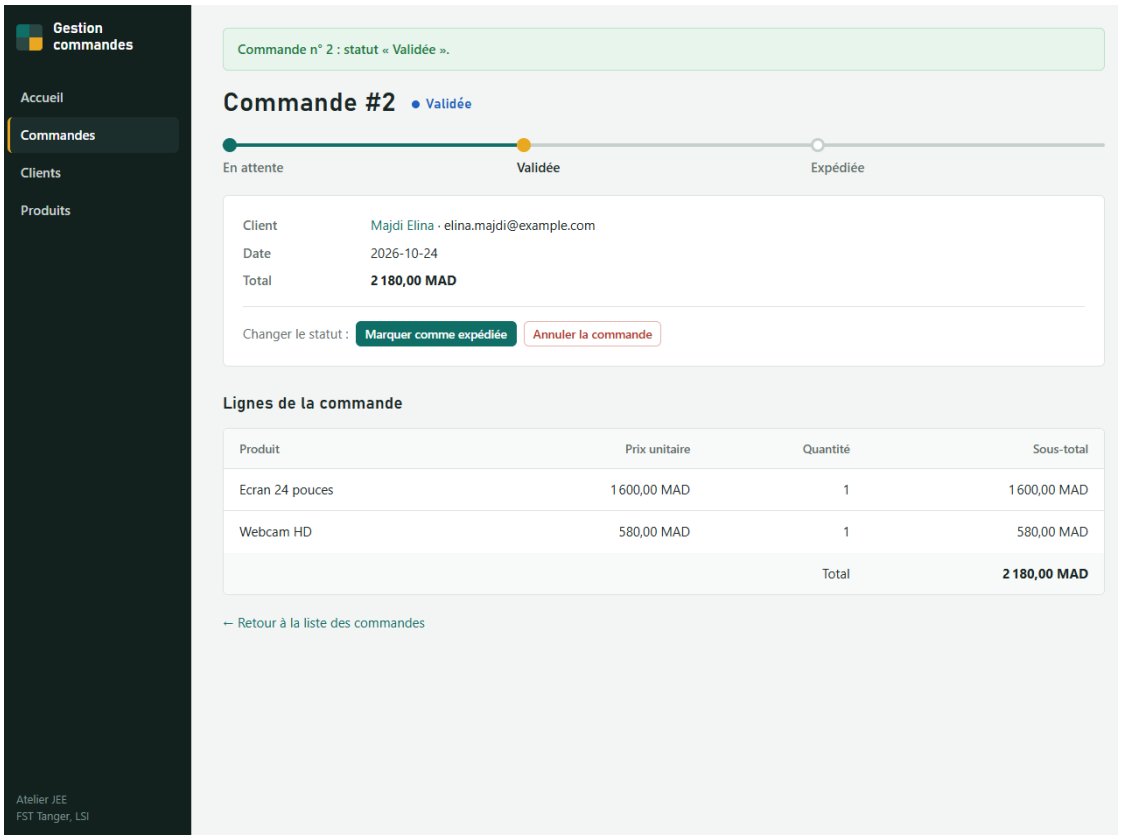


Figure 6.10. Commande validée : lignes verrouillées, expédition possible

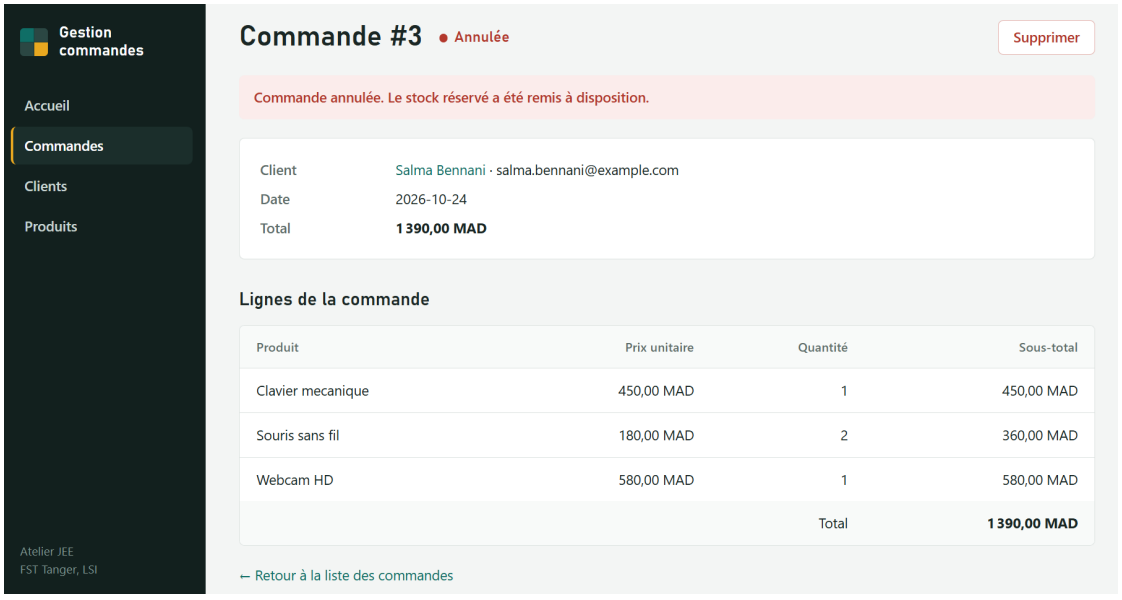


Figure 6.11. Commande annulée

6.5.5 Gestion des erreurs et mobile

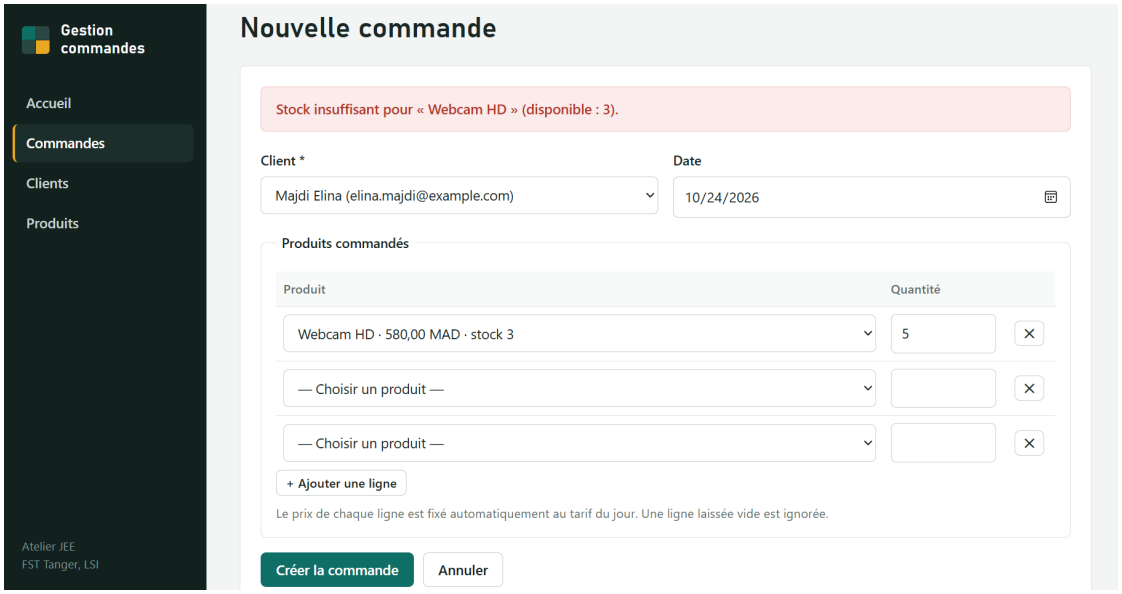


Figure 6.12. Message affiché lorsque la quantité dépasse le stock

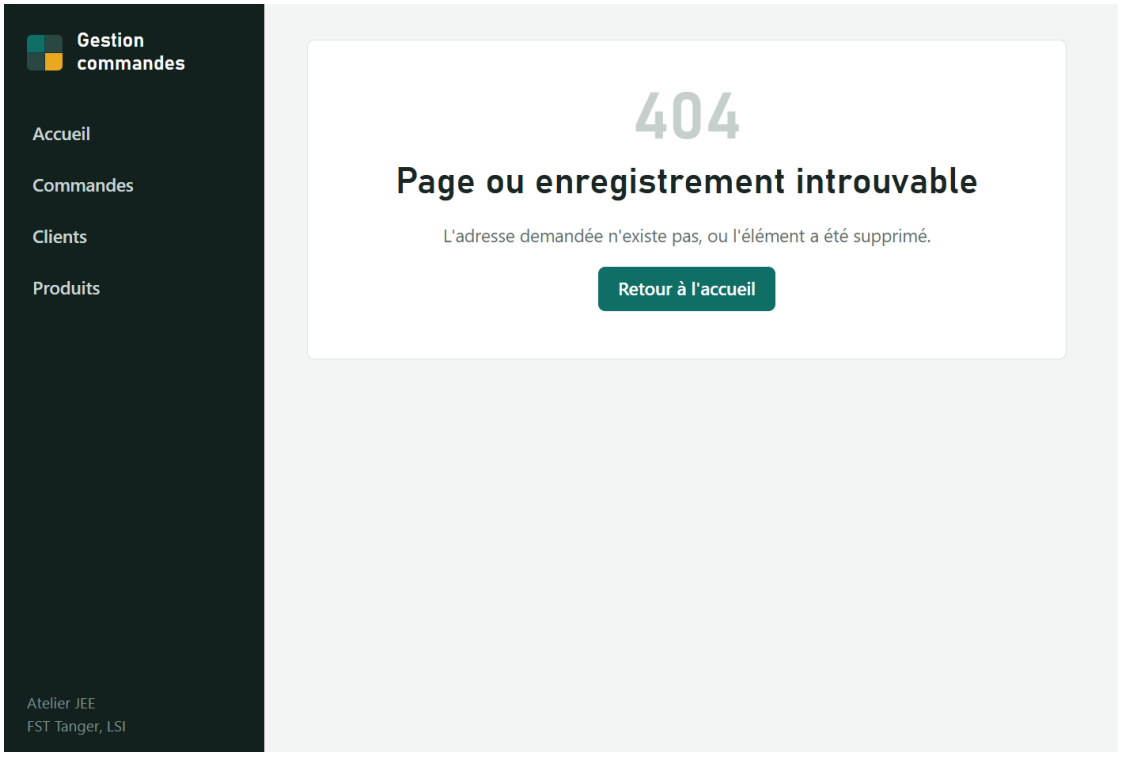


Figure 6.13. Page d'erreur pour une adresse ou un enregistrement inexistant



Figure 6.14. Page d'une commande sur téléphone (largeur 390 px)

Chapitre 7

Validation

7.1 Stratégie

La validation a été menée par scénarios sur l'application déployée, avec des cas nominaux et des cas limites. Chaque règle métier du chapitre 2 a été vérifiée au moins une fois (tableau 7.1).

Table 7.1. Scénarios de validation

Règle	**Scénario**	**Preuve**
Stock réservé	Créer une commande de 2 unités : le stock baisse de 2	Fig. 7.1–7.2
Stock restitué	Annuler cette commande : le stock revient à sa valeur	Fig. 7.3
Stock insuffisant	Commander plus que le stock disponible : refus avec message	Fig. 6.12
Validation des saisies	E-mail invalide : message sous le champ	Fig. 6.4
Contrainte en base	Stock négatif refusé par `CHECK`	Fig. 3.5
Lignes verrouillées	Commande validée : plus de modification de ligne	Fig. 6.10
Adresse inexistante	Page d'erreur propre, sans trace Java	Fig. 6.13

7.2 Preuve de la gestion du stock

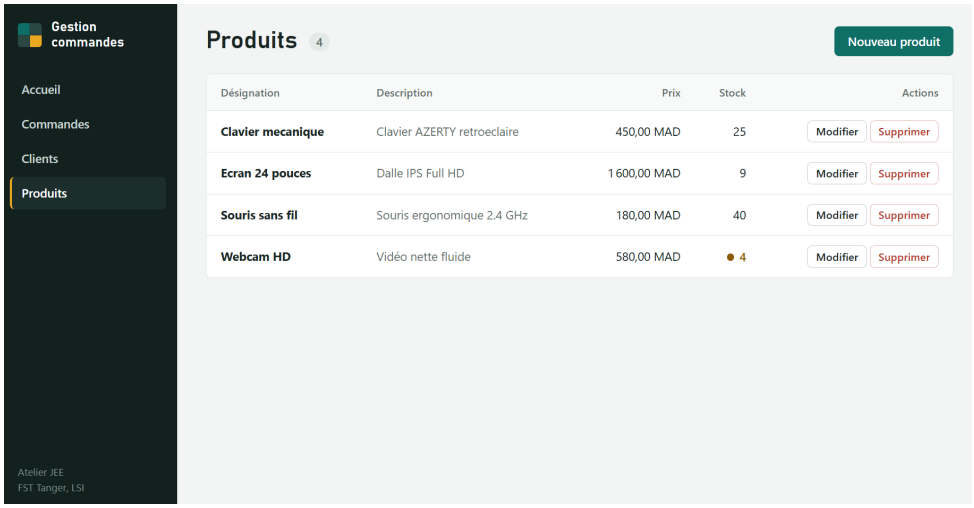


Figure 7.1. Stock avant la commande

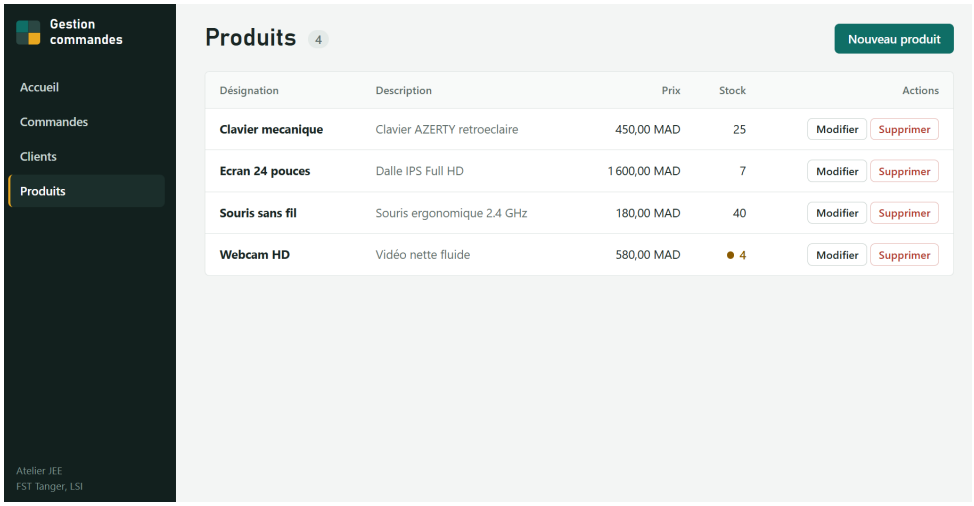


Figure 7.2. Stock après une commande de deux unités

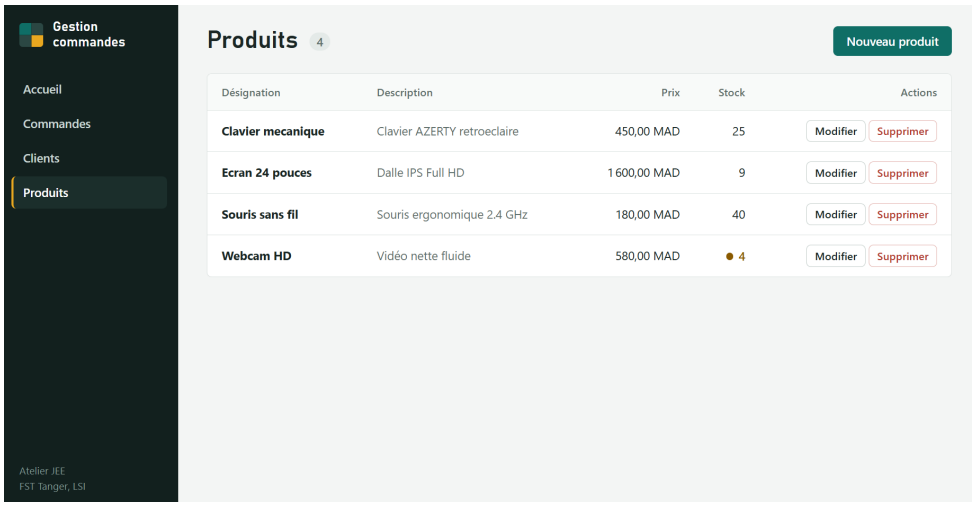


Figure 7.3. Stock après annulation de la commande

7.3 Démarrage et déploiement

Le démarrage du serveur confirme que Weld est actif : la ligne `WELD-ENV-001100` indique que l'injection CDI est disponible dans les servlets et filtres (figure 7.5).

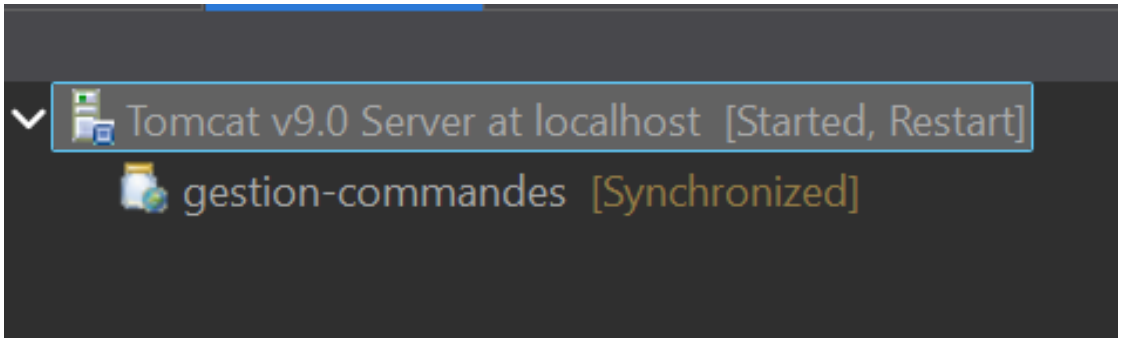


Figure 7.4. Serveur Tomcat démarré dans Eclipse

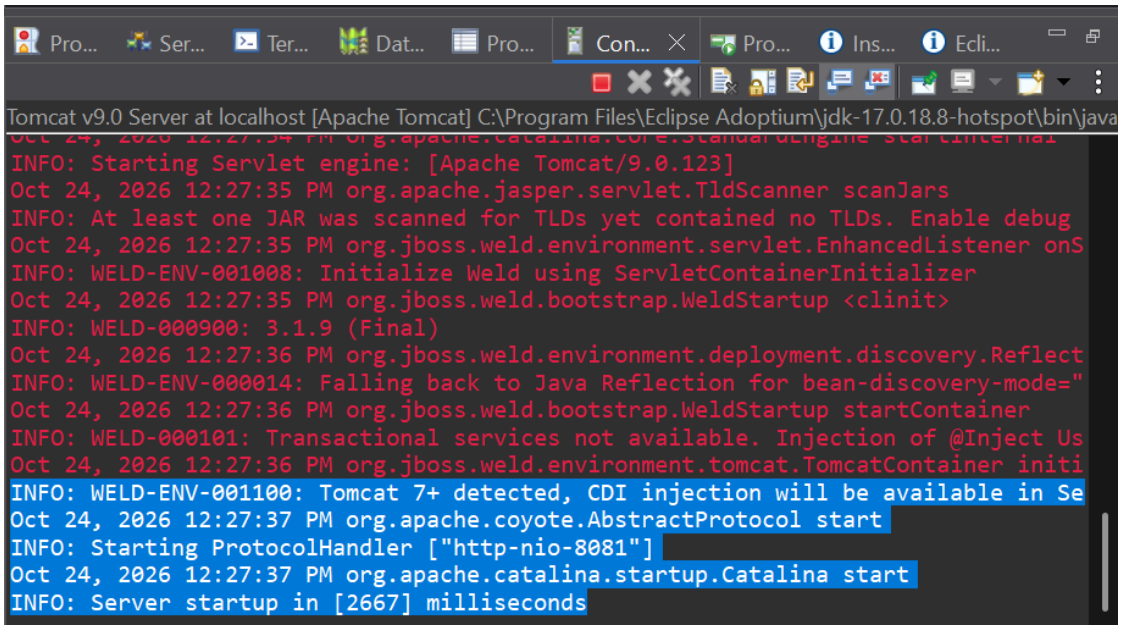


Figure 7.5. Console au démarrage : initialisation de Weld puis démarrage du serveur

Incidents rencontrés. Trois problèmes de configuration ont été résolus pendant le déploiement. Le port 8080 était occupé par un service système Windows : Tomcat a été déplacé sur le port 8081. L'application répondait ensuite par une erreur 404 : Eclipse servait une ancienne copie du projet ; un rafraîchissement, un *Maven Update Project* puis un nettoyage du serveur ont publié les nouveaux fichiers. Enfin, le validateur JSP d'Eclipse refusait l'entité ` ` dans un fichier de balise : elle a été remplacée par ` `.

Chapitre 8

Gestion du code avec Git et GitHub

Le code est versionné dans un dépôt GitHub (`https://github.com/ouhaddousara/jee-order-management-mvc`). Les messages de commit suivent la convention *Conventional Commits* (`feat`, `fix`, `docs`, `style`, `chore`), ce qui rend l'historique lisible : on voit d'un coup d'œil ce qu'apporte chaque étape.

Le fichier `.gitignore` exclut les fichiers générés (`target/`, fichiers Eclipse) et surtout `db.properties`, qui contient le mot de passe de la base. Un modèle `db.properties.example` est fourni à la place.

```
PS C:\dev\gestion-commandes> git log --oneline
cdaec14 (HEAD -> main, origin/main) docs: add README with architectur
e, setup and known limitations
a413054 feat(views): add JSP/JSTL views, tag files, stylesheet and we
b.xml configuration
12020ba feat(web): add servlets, CSRF/UTF-8 filter and flash messages
 for clients, products and orders
aa1f928 fix(dao): name the product and available stock in Insufficien
tStockException
a67b953 feat(dao): add JDBC DAO implementations with transactions and
 stock management
394f9d3 feat(service): add DAO interfaces and persistence exceptions
7f1cced feat(entities): add Client, Produit, Commande, LigneDeCommand
e and StatutCommande
2e4ec52 docs: add UML class diagram (TikZ and Mermaid)
8f5a2ae feat(db): add MySQL schema with constraints and sample data
935b3ee chore: initialize Maven web project (Tomcat 9, Java 17, CDI)
```

Figure 8.1. Historique des commits

```
PS C:\dev\gestion-commandes> git push
Everything up-to-date
```

Figure 8.2. Envoi des commits vers GitHub

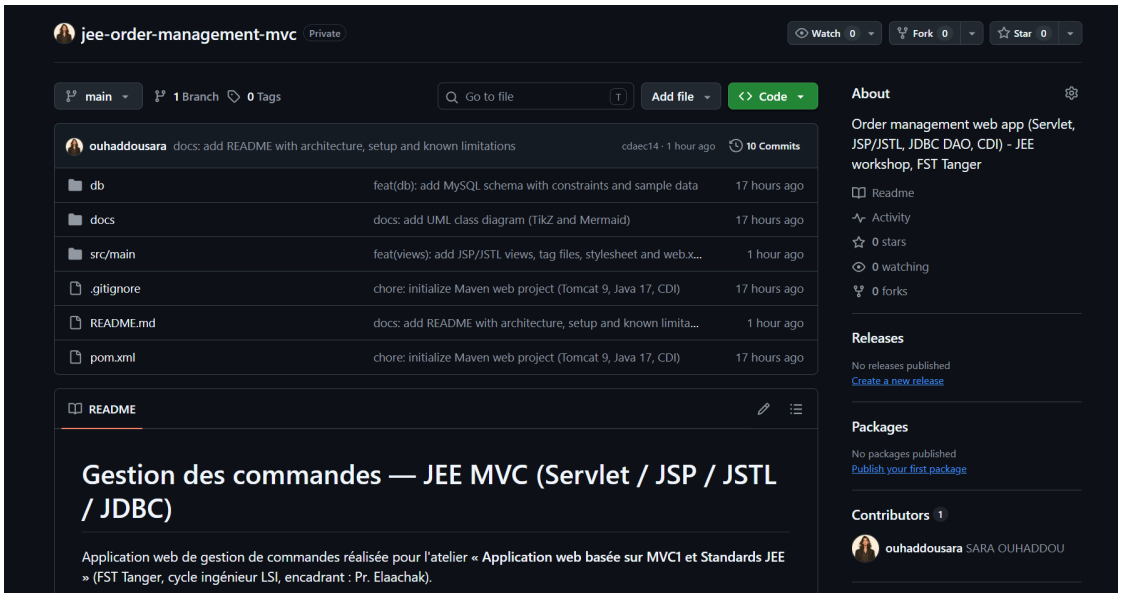


Figure 8.3. Page du dépôt GitHub

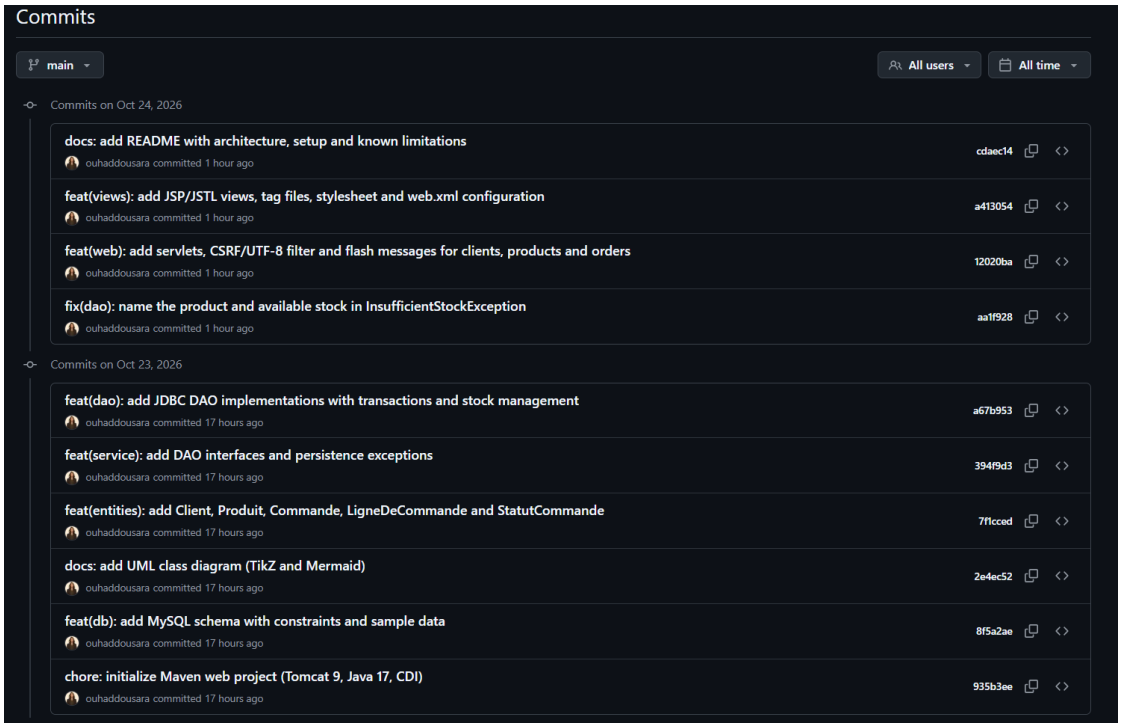


Figure 8.4. Liste des commits sur GitHub

Chapitre 9

Limites et perspectives

Plusieurs choix ont été faits pour rester dans le périmètre de l'atelier. Ils méritent d'être signalés.

— **Pas de pool de connexions** : chaque opération ouvre une connexion via `DriverManager`. C'est suffisant pour un usage pédagogique ; en production, un `DataSource` avec pool (HikariCP, pool de Tomcat) serait nécessaire.

— **Pas d'authentification** : tout utilisateur voit toutes les données. Le jeton CSRF est rattaché à la session, pas à un compte.

— **Modification d'un produit** : l'édition d'un produit écrase le stock avec la valeur saisie. Si une commande passe pendant l'édition, sa réservation de stock est perdue. Une correction possible est de modifier le stock par des mouvements (entrée, sortie) plutôt que par remplacement.

— **Pas de pagination** : les listes affichent tous les enregistrements.

— **Pas de tests automatisés dans le dépôt** : la validation du chapitre 7 est manuelle. Des tests JUnit sur les DAO avec une base de test seraient la suite naturelle.

— **Une seule devise** (MAD) et pas de gestion de TVA ou de remises.

Conclusion

L'atelier a permis de construire de bout en bout une application JEE : du diagramme de classes à une interface fonctionnelle déployée sur Tomcat. Les principales difficultés n'étaient pas dans le CRUD, mais dans la cohérence des données : réserver le stock sans jamais le rendre négatif, figer les prix, imposer un cycle de vie aux commandes. La réponse retenue est de placer chaque règle là où elle peut être garantie, c'est-à-dire dans la base pour les invariants (contraintes, mise à jour conditionnelle du stock), dans les transactions pour les opérations composées et dans les énumérations pour les règles métier.

La séparation en couches (entités, interfaces, DAO, servlets, JSP) rend le code lisible et prêt à évoluer vers un framework comme Spring ou Jakarta EE, dont les concepts (injection, contrôleurs, vues) ont ici été manipulés à la main.

Bibliographie

[1] Pr. L. Elaachak, *Application web basée sur MVC et Standards JEE*, sujet d'atelier, FST Tanger, cycle LSI.

[2] DZone, *Tutorial covering JSP 2.2 and JSTL*, `https://dzone.com/articles/tutorial-covering-jsp-22-and`.

[3] Apache Software Foundation, *Apache Tomcat 9 Documentation*, `https://tomcat.apache.org/tomcat-9.0-doc/`.

[4] JBoss, *Weld Reference Documentation : Servlet containers*, `https://weld.cdi-spec.org/documentation/`.

[5] OWASP, *Cross-Site Request Forgery Prevention Cheat Sheet*, `https://cheatsheetseries.owasp.org/`.

[6] Oracle, *MySQL 8.0 Reference Manual : CHECK constraints*, `https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html`.

Annexe A

Structure du dépôt

```
gestion-commandes/
  db/schema.sql                     schema MySQL + donnees d'exemple
  docs/class-diagram.{md,tex}       diagramme de classes (Mermaid, TikZ)
  src/main/java/ma/fstt/
    entities/                       Client, Produit, Commande, LigneDeCommande, StatutCommande
    service/                        IDao, IClientDao, ... et exceptions
    dao/                            implementations JDBC, DBConnection, Jdbc
    web/                            servlets, RequestFilter, Flash, Web
  src/main/resources/               db.properties.example
  src/main/webapp/
    WEB-INF/{web.xml,beans.xml,tags/,views/}
    assets/{css,js}
  pom.xml
  README.md
```

Listing A.1. Arborescence principale